

# BUGWARS

## GAME PROJECT

**Profesor:**  
Santiago Fazzini

**Autor:**  
Luis Nava 153739

# ESTRUCTURA DEL PROYECTO

**01. Introducción**

**02. Descripción del  
Juego**

**03. Desarrollo del  
proyecto**

**04. Conceptos  
aplicados**

**05. Bibliotecas  
utilizadas**

**06. Pruebas unitarias  
(TDD)**

**07. Manual de uso**

**08. Conclusión**

# 01. INTRODUCCIÓN

El presente trabajo consiste en el desarrollo de un videojuego llamado **BugWars**, realizado como proyecto final de la materia Programación II.

Se trata de un juego en 2D desarrollado en **C++**, utilizando la biblioteca **SFML** para la interfaz gráfica y siguiendo el patrón de arquitectura **MVC** (Modelo, Vista y Controlador)

El objetivo fue aplicar los principales conceptos vistos durante la cursada, priorizando una estructura ordenada, código reutilizable y una correcta separación de responsabilidades.



## Entorno de Compilación

Arquitectura basada en sistemas 2D. Manejo de colisiones por celdas y actualización gráfica en tiempo real.

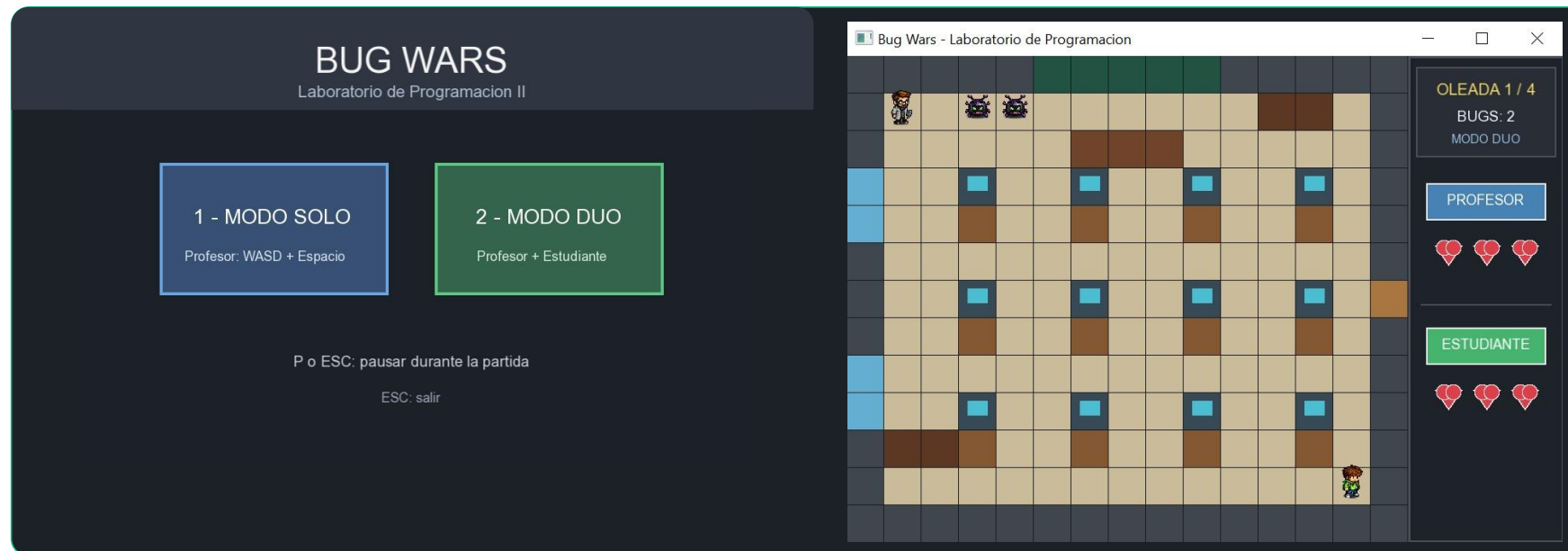
## 02. DESCRIPCIÓN DEL JUEGO

**BugWars** es un juego inspirado en los clásicos arcade de vista superior.

El jugador controla un personaje que debe avanzar por el mapa eliminando enemigos utilizando cargas de código que, luego de un tiempo de compilación, generan una explosión.

Durante la partida aparecen diferentes oleadas de enemigos que intentarán alcanzar al jugador. El objetivo consiste en sobrevivir hasta eliminar todos los enemigos del nivel.

Cada jugador posee un número limitado de vidas y solamente puede tener una carga activa al mismo tiempo.



# HISTORIA Y CONTEXTO

## Temática y Analogía



### Enemigos

Bugs o enemigos de Software



### Bombas

Cargas / Compilaciones de código



### Escenario

Laboratorio de Programación II



### Protagonistas

El Profesor y los Estudiantes

Las **oleadas** simbolizan problemas de programación cada vez más complejos.

Durante una clase en el laboratorio de Programación II con el profesor Santiago Fazzini, una situación inesperada pone en peligro la jornada, una oleada de bugs comienza a salir de las computadoras y toma el control del aula.

El profesor y los estudiantes deberán trabajar juntos para detener la invasión. Para hacerlo, utilizarán cargas de código que, luego de un breve tiempo de compilación, generan explosiones capaces de eliminar a los bugs.

A medida que avanza la partida, aparecen nuevas oleadas con enemigos cada vez más resistentes, aumentando la dificultad del desafío. El objetivo es sobrevivir, limpiar el laboratorio y recuperar el control de las computadoras antes de que los bugs se propaguen por todo el salón.

# 03. DESARROLLO DEL PROYECTO

El proyecto fue desarrollado completamente en lenguaje **C++** utilizando **CodeBlocks** como entorno de desarrollo. Para organizar el código se utilizó el patrón **MVC**.

## Modelo

Contiene toda la lógica del juego. Clases principales:

- Jugador
- Enemigo
- BugBasico
- BugResistente
- SistemaCargas
- CargaCodigo
- Explosion
- Tablero
- Partida

Se implementan las reglas del juego.



## Vista

La vista se encarga únicamente del dibujo del juego.

Se implementó utilizando **SFML** para representar:

- jugadores
- enemigos
- explosiones
- tablero
- menú
- HUD



## Controlador

El controlador recibe las acciones del jugador, actualiza el modelo y coordina la comunicación entre el modelo y la vista.

De esta forma **cada componente tiene una responsabilidad definida.**



## 04. CONCEPTOS APLICADOS

Durante el desarrollo se utilizaron diferentes conceptos vistos en la materia.

### POO

Se aplicó encapsulamiento mediante clases con atributos privados y métodos públicos. También se utilizó composición para construir las diferentes entidades del juego.

### Herencia

La clase **Enemigo** fue utilizada como clase base para implementar distintos tipos de enemigos. De ella derivan: **BugBasico** y **BugResistente** (cada uno con comportamiento propio).

### Containers STL

Se utilizaron estructuras de la STL (principalmente **std::vector** y **std::array**). También se emplearon algoritmos de `<algorithm>` para eliminar elementos y recorrer colecciones.

### Arquitectura MVC

Toda la aplicación fue organizada utilizando el patrón Modelo – Vista – Controlador, separando la lógica del juego del renderizado y del manejo de entradas.

## 05. BIBLIOTECAS UTILIZADAS

Para desarrollar la interfaz gráfica y comprobar la lógica del Modelo se utilizaron las bibliotecas SFML 3 y Tau.

### SFML 3

Utilizada para:

- Renderizado
- Manejo de ventanas
- Entrada de teclado
- Imágenes
- Textos

### Tau

[Framework](#) utilizado para realizar las pruebas unitarias del modelo



```
#include <tau/tau.h>
TAU_MAIN() // sets up Tau (+ main function)

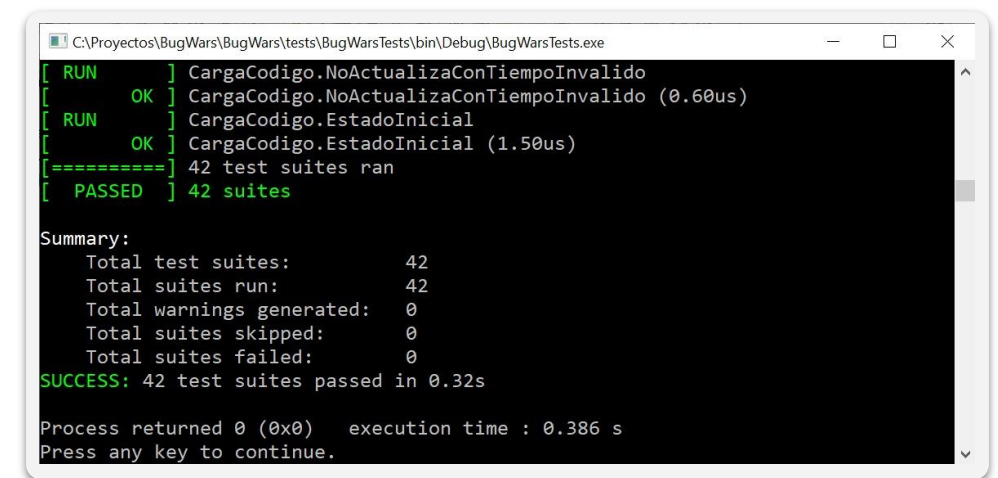
TEST(foo, bar1) {
    int a = 42;
    int b = 13;
    CHECK_GE(a, b); // pass :)
    CHECK_LE(b, 8); // fail - Test suite not aborted
}

TEST(foo, bar2) {
    char* a = "foo";
    char* b = "foobar";
    REQUIRE_STREQ(a, a); // pass :)
    REQUIRE_STREQ(a, b); // fail - Test suite aborted
}
```

## 06. PRUEBAS UNITARIAS (TDD)

Para verificar el funcionamiento de la lógica del juego se implementaron pruebas unitarias utilizando el framework Tau. Las pruebas fueron realizadas sobre las clases pertenecientes al modelo, ya que contienen la mayor parte de la lógica de negocio.

| Clase Evaluada | Cantidad de pruebas         | Estado final                     |
|----------------|-----------------------------|----------------------------------|
| Jugador        | 6 Pruebas                   | ✓ Correcto                       |
| CargaCodigo    | 5 Pruebas                   | ✓ Correcto                       |
| Explosion      | 6 Pruebas                   | ✓ Correcto                       |
| Tablero        | 12 Pruebas                  | ✓ Correcto                       |
| SistemaCargas  | 13 Pruebas                  | ✓ Correcto                       |
| <b>TOTAL</b>   | <b>42 Pruebas Unitarias</b> | <b>FINALIZARON CORRECTAMENTE</b> |



```

C:\Proyectos\BugWars\BugWars\tests\BugWarsTests\bin\Debug\BugWarsTests.exe
[ RUN      ] CargaCodigo.NoActualizaConTiempoInvalido
[ OK       ] CargaCodigo.NoActualizaConTiempoInvalido (0.60us)
[ RUN      ] CargaCodigo.EstadoInicial
[ OK       ] CargaCodigo.EstadoInicial (1.50us)
[=====  
[ PASSED  ] 42 suites
Summary:
  Total test suites:      42
  Total suites run:      42
  Total warnings generated: 0
  Total suites skipped:  0
  Total suites failed:   0
SUCCESS: 42 test suites passed in 0.32s
Process returned 0 (0x0)   execution time : 0.386 s
Press any key to continue.
  
```

# 07. MANUAL DE USO Y CONTROLES



## 08. CONCLUSIÓN

### Integración Académica

Este trabajo permitió integrar los principales contenidos desarrollados durante la materia.

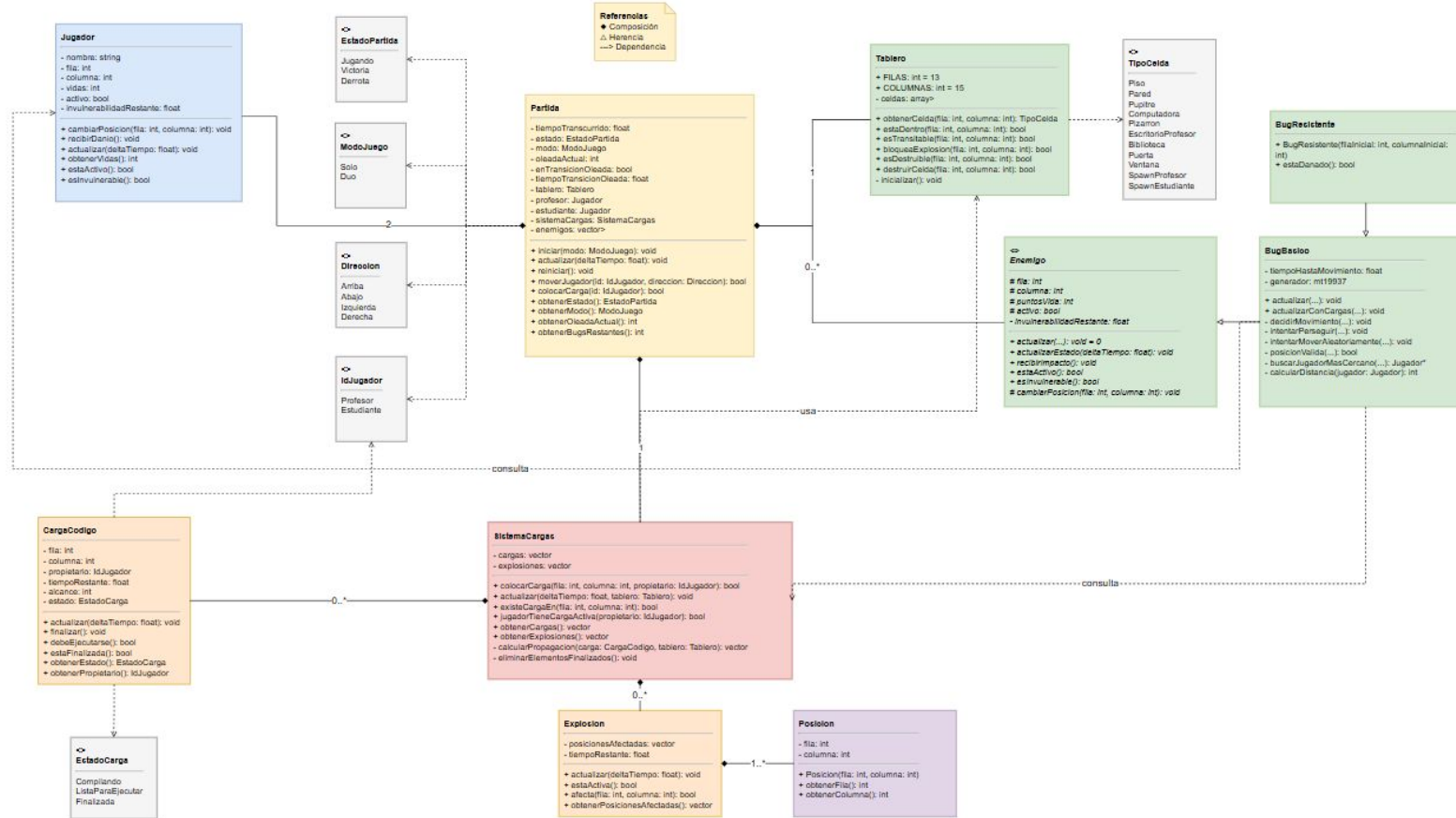
Además del funcionamiento del juego, se priorizó la organización del código, la aplicación de buenas prácticas de programación y la validación de la lógica mediante pruebas unitarias.

### Resultado obtenido

El resultado es un proyecto organizado utilizando **MVC**, desarrollado en **C++** y acompañado por un conjunto de **42 pruebas unitarias** que verifican el comportamiento de las clases principales del modelo.

# Anexo UML

Diagrama de Clases UML - Modelo de Bug Wars



[Link Draw.io](https://www.draw.io)

**¡GRACIAS!**